

DISTINCT

Cross-Network Document Transfer Platform

Development Plan & Delivery Roadmap

.NET 10

RabbitMQ 4.3

PostgreSQL 16

OpenShift

OpenTelemetry

One platform, five milestones, four people

21

weeks · 4 Program Increments

5

delivery milestones

4

engineers — 2 backend, 1
automation, 1 DevOps

Sprint 3

first end-to-end flow live

Distinct moves documents between two segregated networks through a sanitizing Proxy, converting and splitting files per-format on the way out and reassembling them on the way in. This plan rebuilds the reliability core around a Postgres-native claim / heartbeat / outbox pattern in place of the current Temporal-based POC, proves out the simplest possible end-to-end path first, and then layers in format conversion, per-format autoscaling, VIP prioritization, and full OpenTelemetry observability — with every engineer pairing with Claude Code to keep a four-person team moving on parallel tracks across two networks and a platform layer.

Flagship Milestone 1: a plain .txt file, submitted once, travels Network A → Proxy → Network B and back with zero conversion or splitting logic in the way — proving the reliability core before any transformation complexity is added.

What ships first, and what deliberately waits

IN SCOPE FOR V1

- ✓ Multi-format ingestion, per-format split & convert
- ✓ Reassembly + reverse-convert in Network B
- ✓ Per-client folder
- ✓ VIP-client priority queueing
- ✓ CSV reporting
- ✓ OpenTelemetry metrics & distributed tracing
- ✓ Crash-safe reliability core on OpenShift

OUT OF SCOPE FOR V1

- End-user facing UI
- General-purpose logging UI / log search
- SLA tiers beyond client-provided values
- Format or feature requirements not yet specified by the client

Engineering assumptions that shape how we build

1 Spec-Driven Development (SDD) with BMAD

Every feature starts as a structured spec — requirements, data model, contracts, quickstart — before implementation, worked through the BMAD-METHOD's agent roles (Analyst → PM → Architect → Dev → QA) with Claude Code. Adoption ramps in parallel with the team's existing workflow to manage the learning curve.

2 OpenTelemetry From the First Deliverable

Metrics and traces are wired into Milestone 1's thin slice, not bolted on later. Every subsequent milestone extends the same instrumentation instead of retrofitting it — observability is a day-one requirement, not an afterthought bolted on at go-live.

3 Chaos Testing Is Continuous

Fault-injection and crash-recovery tests run in CI from Milestone 2 onward, not as a one-time validation gate. Every change to the reliability core — claim/heartbeat, sweeper, outbox — is proven against worker crashes before it ships.

4 AI Engineering — Design Through Implementation

Claude Code is used across the full lifecycle, not just as a coding assistant bolted on at the end. Architecture trade-offs, spec authoring, implementation, code review, and test generation all run through the same AI-assisted loop, so design and build stay in lockstep instead of handing off between them.

Five milestones across four Program Increments



Milestones can overlap by design — reliability hardening (M2) and per-format scaling (M3) share sprint boundaries, and M3 is still finishing as M4 begins. The detailed Gantt on a later slide shows exact task-level dependencies.

Prove the full path end-to-end before adding any transformation complexity

WHAT'S DELIVERED

- Ingestion API accepts a job containing plain .txt files
- Routing / decision node recognizes “no conversion or split needed” and takes the pass-through path
- Reliability Core (claim → complete + transactional outbox) live for this thin slice
- Proxy Listener → Assembly → Reverse-Converter (no-op) → Reporting Worker → callback, deployed to OpenShift dev
- Outbox Relay hardened for the post-commit-crash path
- Mini-manifest-written-before-commit pattern on Network A
- Priority queue infrastructure in RabbitMQ 4.3
- VIP-client tagging end-to-end, from ingestion through reporting
- Per-client SLA and timeout ingestion from client-provided configuration
- Baseline internal OpenTelemetry metrics and traces across the whole path

SUCCESS CRITERIA / DEMO

A .txt job submitted to Network A is reassembled, reported (CSV), and acknowledged back to Network A.

A VIP job submitted alongside a backlog of standard jobs is processed ahead of the queue, and per-client SLA/timeout values are enforced.

Observability end-to-end in OpenTelemetry.

Demoed live at the Sprint 3 review.

Make the reliability core survive real failures

WHAT'S DELIVERED

- Sweeper service for stale, crashed-worker job recovery
- Worker exception framework: transient retries via TTL/DLX, permanent failures via dead queue and DLQ consumers
- Mini-manifest-written-before-commit pattern on Network A
- Job status rollup (Completed / CompletedPartially / Failed) and a 30-minute timeout sweeper
- Chaos test suite that kills workers mid-job and asserts recovery

SUCCESS CRITERIA / DEMO

Killing a worker at any point in the pipeline — mid-claim, post-commit — results in automatic recovery within the design SLAs (~90s via the sweeper, ~500ms via the outbox relay), proven by automated chaos tests running in CI.

Add real conversion and splitting without letting heavy formats stall fast ones

WHAT'S DELIVERED

- Docx, image, and media splitters/converters split out as independent per-format microservices
- Per-format horizontal pod autoscaling on OpenShift
- Full 5-subflow Proxy Listener: out-of-order chunk/manifest buffering, inline drain, and error sentinels
- Per-client folder
- Linear-vs-decision-tree routing benchmark with a data-backed recommendation

SUCCESS CRITERIA / DEMO

A mixed batch of PDF, DOCX, image, and media files completes with fast formats unaffected by a backlog of slow ones, and chunk/manifest arrival order is proven not to matter to the outcome.

Production-harden the platform and cut over

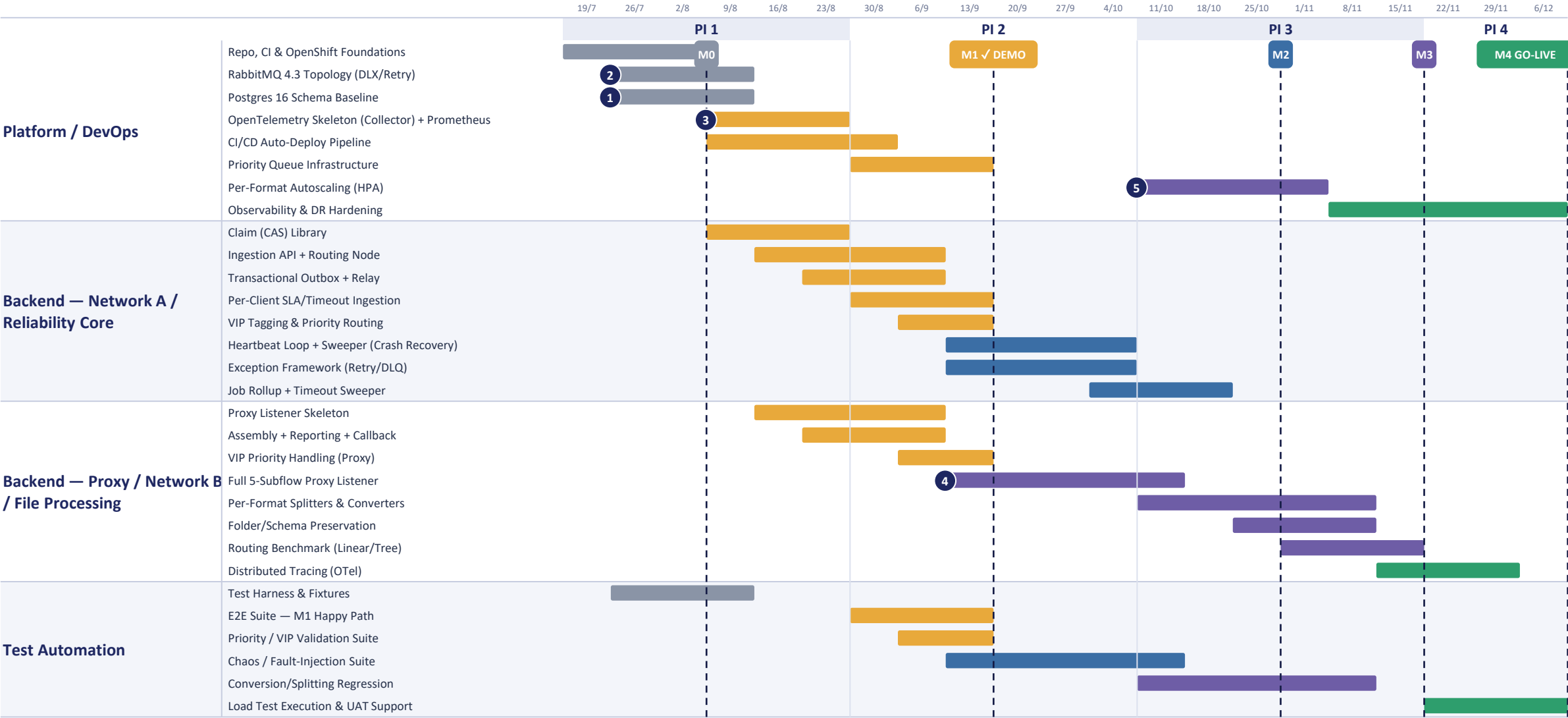
WHAT'S DELIVERED

- Full OpenTelemetry metrics/traces/logs pipeline with dashboards and alert thresholds
- Load and performance testing at target monthly job volume

SUCCESS CRITERIA / DEMO

The platform sustains target monthly job volume within SLA under load test, alerts fire correctly during a DR drill, and UAT is signed off by stakeholders ahead of go-live.

Sprint & PI Gantt — 21 Weeks Across 4 Program Increments



① Postgres schema baseline gates the Reliability Core claim library and later heartbeat/sweeper work (Backend A)
② RabbitMQ topology gates the Outbox Relay and Proxy Listener work (Backend A/B)
③ OpenTelemetry skeleton gates all later per-service instrumentation, through M4

④ Full Proxy Listener depends on Sweeper/Outbox hardening delivered in M2
⑤ Per-format autoscaling depends on the per-format microservice split completing in M3

What could slow this down, and how we'll handle it

RISK	SEVERITY	MITIGATION
SDD Learning Curve	Medium	SDD learning and adoption is done in parallel with the “traditional” way of work.
Heavy-format contention if the per-format microservice split slips	Medium	Land the per-format split as the first task of Milestone 3, not the last.
Client-provided inputs (SLA, timeout) arrive late and block M3	Medium	Request formally at kickoff with a named owner and due date; use documented defaults if missed.